Toward Optimizing Networking and Cybersecurity Applications Using Domain-Specific Accelerators for Dynamic Programming

Ali Mazloum*, Samia Choueiri*, Sergio Elizalde*, Ali AlSabeh†, Elie F. Kfoury*, Nasir Ghani‡, Jorge Crichigno*
* Molinaroli College of Engineering and Computing, University of South Carolina, Columbia, U.S.A.
† College of Sciences and Engineering, University of South Carolina Aiken, Aiken, U.S.A.
‡Department of Electrical Engineering, University of South Florida, Fowler Ave Tampa, U.S.A.
*amazloum@email.sc.edu, *choueiri@email.sc.edu, *elizalds@email.sc.edu, †ali.alsabeh@usca.edu,
*ekfoury@email.sc.edu, ‡nghani@usf.edu, *jcrichigno@cec.sc.edu

***Abstract*—Dynamic Programming Accelerators (DPAs) provide domain-specific hardware support for accelerating dynamic programming (DP) workloads through specialized instructions and massive parallelism. These capabilities make DPAs a promising approach for improving the efficiency of compute-intensive functions in network management systems, such as routing optimization, failure recovery, and traffic analysis, where timely processing is critical for maintaining performance and reliability. To explore this potential, this paper considers two representative use cases. First, network resilience is formulated as an all-pairs shortest path (APSP) problem using the Floyd–Warshall algorithm, enabling fast recomputation of routing paths in response to link or node failures. Second, deep packet inspection (DPI) is formulated as a sequence alignment problem using the Smith–Waterman algorithm, supporting both partial matching and regular expression semantics for detecting complex and evolving traffic patterns in packet payloads.**

The proposed approach is evaluated through a comprehensive experimental study considering key performance metrics, including processing time, computational throughput, and energy consumption. The evaluation compares DPA-enabled implementations against general-purpose graphics processing units (GPGPUs) and CPUs across a range of problem sizes. The results indicate that DPAs can significantly reduce computation time, achieve higher throughput, and improve energy efficiency for DP-based workloads in network management.

***Index Terms*—Network management, network resilience, deep packet inspection (DPI), dynamic programming (DP), domain-specific accelerator (DSA), dynamic programming accelerator (DPA), CUDA.**

I. Introduction

DYNAMIC programming (DP) is a fundamental optimization technique widely used to solve complex problems by decomposing them into smaller, overlapping subproblems. By reusing intermediate results, DP reduces computational redundancy and enables efficient solutions to problems that would otherwise be intractable. In networking, DP has been applied to a variety of applications, including routing optimization, failure recovery, and traffic analysis [1], where optimal decisions must be computed under strict performance constraints. In the context of network resilience, DP-based formulations can be used to recompute routing paths after failures, providing a systematic approach for deriving alternative paths. However, as network scale and traffic volume continue to grow, the computational demands of DP-based algorithms increasingly exceed the capabilities of general-purpose central processing units (CPUs), which are inherently limited in exploiting fine-grained parallelism [2].

To address these limitations, general-purpose graphics processing units (GPGPUs) have been widely adopted to accelerate DP workloads by leveraging massive thread-level parallelism and high memory bandwidth [3]–[5]. While GPGPUs improve performance compared to CPUs, they remain constrained by the need to emulate DP operations using general-purpose instructions, which may limit efficiency for DP-intensive kernels. Recent advances in GPU architectures have introduced domain-specific support for DP through Dynamic Programming eXecution (DPX) instructions, enabling Dynamic Programming Accelerators (DPAs) [6]. These hardware accelerators provide fused operations tailored to DP recurrences, reducing instruction overhead, improving data locality, and increasing computational throughput [7]. In this work, GPUs supporting DPX instructions are referred to as DPAs.

Despite these architectural advancements, the application of DPAs to network management and cybersecurity workloads remains relatively unexplored. This is particularly relevant in network resilience, where timely failure recovery can impact service availability and quality of experience. As modern networks grow in size and complexity, link and node failures become more frequent [8], triggering reconvergence processes that may take tens of seconds in traditional control planes [9]. During this interval, forwarding inconsistencies and temporary connectivity loss may degrade network performance, especially for latency-sensitive and real-time applications. Accelerating the computation of alternative routing paths can help reduce recovery time and improve overall network robustness, suggesting that DPAs may provide an effective approach to enhancing control plane responsiveness [10].

To demonstrate the broader potential of domain-specific acceleration in network management, this work considers two representative use cases: network resilience and deep packet inspection (DPI). Network resilience is formulated as an all-pairs shortest path (APSP) problem, enabling recomputation of routing paths following failures. In parallel, DPI is modeled as a sequence alignment problem, enabling high-throughput anal-

This work was supported by the U.S. National Science Foundation, Office of Advanced Cyberinfrastructure, Award #2346726.

ysis of packet payloads and supporting approximate matching techniques that improve robustness against evolving traffic patterns [11]–[13]. The source code of the two applications is available in [14].

The proposed approach is evaluated through a comprehensive experimental study that considers key performance metrics, including processing time, computational throughput, and energy consumption. The evaluation compares DPA-enabled implementations against both GPGPU and CPU baselines across a range of problem sizes. The results highlight the potential benefits of DPAs in accelerating DP-based workloads. In particular, APSP can be computed in a few milliseconds for topologies with 3,000 nodes, while processing 100,000 signatures requires only hundreds of microseconds using partial matching, and a few milliseconds when supporting the more expressive regex-based matching.

The remainder of this paper is organized as follows. Section II describes the motivation and contributions. Section III provides background on the GPU programming model, CUDA, and DPA. Section IV presents the related work. Section V discusses the two systems proposed by this work, the network resilience application and the DPI application. Section VI presents the experimental results, and Section VII concludes the paper.

II. Motivation and Contributions

The slowdown of Moore's law [15], [16] and the end of Dennard scaling have driven a fundamental shift in processor architecture, from single-core designs to multi-core and many-core systems [17]. This transition marked the early stages of parallel programming, where performance improvements could no longer be achieved solely through hardware scaling. Instead, developers were required to restructure applications to exploit parallelism. However, the benefits of parallelization are inherently limited, as described by Amdahl's law [18], which bounds the achievable speedup based on the sequential portion of a workload.

These limitations have motivated the development of domain-specific accelerators (DSAs), which target particular classes of workloads to achieve higher efficiency [19]. In networking, programmable data plane switches and P4 [20]–[23] exemplify DSAs designed to accelerate packet processing and network management. More recently, additional accelerators have emerged to support various network functions, such as smart network interface cards that integrate specialized engines for cryptography, compression, and regex. These advances have primarily focused on accelerating data plane operations. In contrast, efforts to improve control plane performance have largely relied on software-based optimizations, such as DPDK [17]. The emergence of DPAs provides a new opportunity to explore hardware acceleration for control plane functions, particularly since many network management tasks can be formulated as dynamic programming problems.

To investigate this opportunity, this work considers two representative networking management applications that can be partially or fully executed within the control plane and evaluates the benefits of implementing them using DPAs. The main contributions of this paper are summarized as follows:

- Demonstrating the use of DPAs to accelerate two fundamentally different network management applications: network resilience (failure recovery) and DPI (traffic analysis).
- Evaluating performance across multiple metrics, including processing time, computational throughput, and energy consumption, and comparing against GPGPU and CPU baselines.
- Analyzing the impact of domain-specific acceleration on control plane efficiency, highlighting the potential of DPAs to enhance network management operations.
- Providing detailed implementation insights and releasing the source code to support reproducibility [14].

III. Background

A. GPU Programming Model

Modern NVIDIA GPUs follow the Single Instruction Multiple Threads (SIMT) execution model, which extends the Single Instruction Multiple Data (SIMD) paradigm to large-scale multithreading. Under SIMT, groups of threads execute the same instruction concurrently, with execution organized into warps of 32 threads that serve as the minimum scheduling unit [24]. As threads within a warp execute the same set of instructions, the overall processing time of the warp is dominated by the thread with the longest processing time.

A GPU is composed of multiple Streaming Multiprocessors (SMs), each containing several processing cores. Threads within an SM share resources such as registers, caches, and shared memory, enabling efficient data reuse and coordination. The GPU memory hierarchy consists of several levels with different latency and bandwidth characteristics, including high-latency global memory, an L2 cache shared across all SMs, and low-latency on-chip L1/shared memory local to each SM. The L1 and shared memory occupy the same physical space and can be configured by the programmer to optimize data locality and access patterns [25]. Efficient utilization of this hierarchy is critical for achieving high performance in memory-bound applications such as dynamic programming.

B. CUDA and DPA

CUDA is NVIDIA's parallel programming platform that extends C/C++ with constructs for expressing fine-grained parallelism on GPUs [26]. It provides abstractions for organizing threads into blocks, which are further grouped into grids. A kernel is a function executed concurrently by a large number of threads, where each thread is assigned a unique index within its block and grid. This hierarchical execution model allows programmers to control parallelism, synchronization, and memory usage.

Recent GPU architectures introduce domain-specific support for accelerating dynamic programming workloads through Dynamic Programming eXecution (DPX) instructions [6]. These instructions are exposed through low-level CUDA intrinsics and enable fused operations that combine addition, comparison, and selection into a single instruction. Such fusion reduces instruction count and execution latency for dynamic programming kernels.

DPX instructions are supported in GPU architectures with compute capability 9.0 and above, such as the NVIDIA Hopper architecture. The H100 GPU, based on Hopper, includes 132 SMs, a 50 MB L2 cache, and up to 80 GB of high-bandwidth memory with a peak bandwidth of 3.35 TB/s. Each SM is equipped with FP32 CUDA cores, Tensor Cores, and configurable L1/shared memory, providing a powerful platform for both general-purpose and domain-specific acceleration. When DPX support is unavailable, these operations are emulated in software, resulting in reduced performance.

By combining massive thread-level parallelism with specialized DPX instructions, DPAs enable efficient execution of dynamic programming algorithms that are otherwise limited by computational and memory constraints on conventional architectures.

IV. Related Work

A wide range of approaches has been proposed to address the network resilience problem. He et al. [27] introduced a main and secondary controller assignment strategy to mitigate the impact of multiple failures. Hirano et al. [28] proposed a network design methodology aimed at minimizing congestion under multiple link failures. Cheng et al. [29] investigated resilient service placement and workload allocation through a two-stage adaptive robust optimization model that captures uncertain node failures and resource demands. While modeling network resilience as an APSP problem is not a widely adopted approach [10], shortest path formulations have been widely used in related contexts, such as network augmentation and service chain construction [30].

Prior work on accelerating APSP computations, particularly the Floyd–Warshall algorithm, has primarily focused on GPU-based optimizations. Several studies have proposed parallel implementations using block decomposition, tiling, and shared memory techniques to improve data locality and reduce global memory access overhead [31]–[33]. These approaches exploit the regular structure of the Floyd–Warshall recurrence to enable fine-grained parallelism across matrix elements, often achieving substantial speedups over CPU baselines. However, such implementations are typically optimized at the CUDA kernel level and require careful tuning of the memory hierarchy and thread organization to achieve high performance.

More recent advances in GPU architectures have introduced domain-specific support for dynamic programming through DPX instructions. Luo et al. [7] showed that these instructions can improve throughput and reduce latency for DP-intensive kernels compared to conventional GPU implementations. Despite these developments, the use of DPX-enabled GPUs for APSP problems in networking contexts remains limited. In particular, prior work has not explored their use for accelerating Floyd–Warshall-based recomputation under dynamic network conditions.

In parallel, GPU-accelerated deep packet inspection (DPI) has been extensively studied, with early work primarily focusing on exact pattern matching. Algorithms such as Aho–Corasick (AC) [34] have been widely adopted due to their efficiency in multi-pattern matching. Systems like Gnort [35] demonstrated the benefits of offloading AC-based pattern matching to GPUs, while subsequent work improved performance through parallelization strategies and memory access optimizations [36], [37]. The PFAC algorithm [38] further enhanced parallelism by eliminating failure transitions, enabling independent matching from multiple input positions, with later improvements targeting memory efficiency [39].

However, exact matching approaches are inherently limited in detecting modified or obfuscated signatures. To address this limitation, sequence alignment techniques such as Needleman–Wunsch and Smith–Waterman have been explored for approximate matching, particularly in malware detection [11], [12]. While these methods demonstrate high detection accuracy, their application has largely been restricted to offline analysis rather than real-time DPI. Furthermore, prior work has not explored the use of domain-specific accelerators for sequence alignment in DPI or their applicability to networking contexts.

This paper addresses these gaps by leveraging domain-specific accelerator support for dynamic programming to accelerate both network resilience and partial-matching DPI. By utilizing accelerator-aware optimizations, the proposed approach enables faster recomputation under dynamic network conditions and more robust detection of mutated signatures, bridging the gap between high-performance computing techniques and practical network management applications.

V. Proposed Systems

A. Network Resilience

This section introduces the Floyd–Warshall algorithm and formulates the network resilience problem as an APSP problem. It then presents the CUDA-based implementation, including the memory access design.

1) Floyd–Warshall Algorithm: The Floyd–Warshall algorithm is a dynamic programming approach for computing shortest paths between all pairs of vertices in a weighted graph. It determines the minimum cost path between every pair of vertices i and j, and supports graphs with negative edge weights, provided that no negative cycles are present. Let V denote the set of vertices with $|V| = n$, and let $w(i, j)$ represent the weight of the edge from vertex i to vertex j, or ∞ if no such edge exists. Define a distance matrix $D \in \mathbb{R}^{n\times n}$, where $D^{(k)}_{i,j}$ represents the shortest path distance from i to j using only the intermediate vertices in the set $\{1, 2, \ldots, k\}$.

The matrix is initialized as:

$$D^{(0)}_{i,j} = \begin{cases} 0 & \text{if } i = j \\ w(i,j) & \text{otherwise} \end{cases}$$

It is then iteratively updated for $k = 1, \ldots, n$ as:

$$D^{(k)}_{i,j} = \min\left(D^{(k-1)}_{i,j},\ D^{(k-1)}_{i,k} + D^{(k-1)}_{k,j}\right) \tag{1}$$

This recurrence reflects that the shortest path from i to j either excludes vertex k or passes through it as an intermediate node.

After completing all iterations, the matrix $D^{(n)}$ contains the shortest path distances between all vertex pairs. Negative cycles can be detected if any diagonal entry satisfies $D^{(n)}_{i,i} < 0$. Path reconstruction can be supported by maintaining a predecessor matrix throughout the computation.

Algorithm 1 Serialized Floyd–Warshall Algorithm with Path Reconstruction.

Require: Vertex set $V = \{v_0, v_1, \ldots, v_n\}$, edge set $E = \{e_{i,j}\}$

Initialize $D_{i,j} \leftarrow w(i,j)$ for all $i, j \in V$
Initialize $P_{i,j} \leftarrow \begin{cases} i & \text{if } i \neq j \text{ and } w(i,j) < \infty \\ \text{NIL} & \text{otherwise} \end{cases}$
for each intermediate vertex $v_k \in V$ **do**
  for each source vertex $v_i \in V$ **do**
    for each destination vertex $v_j \in V$ **do**
      if $D_{i,k} + D_{k,j} < D_{i,j}$ **then**
        $D_{i,j} \leftarrow D_{i,k} + D_{k,j}$
        $P_{i,j} \leftarrow P_{k,j}$
      end if
    end for
  end for
end for
return D, P

2) Problem Formulation: The network topology is modeled as a weighted graph $G = (V, E)$, where $V = \{v_0, v_1, \ldots, v_n\}$ represents network devices (e.g., routers and switches), and $E = \{e_{i,j} \mid v_i, v_j \in V\}$ represents the set of links. Each edge $e_{i,j}$ is associated with a weight corresponding to a cost metric such as latency or physical distance.

Let $D^{(n)} \in \mathbb{R}^{n \times n}$ denote the APSP distance matrix, where each entry $D^{(n)}_{i,j}$ represents the minimum path cost from node v_i to node v_j. Similarly, let $P^{(n)} \in \mathbb{Z}^{n \times n}$ denote the predecessor matrix, where $P^{(n)}_{i,j}$ stores the immediate predecessor of v_j along the shortest path from v_i.

Dynamic network conditions are considered in the form of node and link failures. A node failure at $v_f \in V$ updates the topology as $V \leftarrow V \setminus \{v_f\}$ and $E \leftarrow E \setminus \{e_{i,f}, e_{f,i} \mid v_i \in V\}$. Similarly, a link failure between nodes v_i and v_j is modeled as $E \leftarrow E \setminus \{e_{i,j}, e_{j,i}\}$. Following such changes, the matrices $D^{(n)}$ and $P^{(n)}$ must be recomputed to reflect updated shortest paths.

3) CUDA Implementation: The Floyd–Warshall algorithm consists of three nested loops over the vertex set V, as shown in Algorithm 1. In each iteration of the outer loop, a vertex $v_k \in V$ is selected as the intermediate node. The inner loops iterate over all source-destination pairs $(v_i, v_j) \in V \times V$, updating $D_{i,j}$ according to Equation 1. Each update depends only on $D_{i,k}$, $D_{k,j}$, and the current value of $D_{i,j}$.

For a fixed intermediate vertex v_k, all updates to $D_{i,j}$ are independent and can therefore be executed in parallel. In the CUDA implementation, the inner two loops are replaced by a GPU kernel that computes the entire matrix $D^{(k)}$ in a single launch. When the number of available GPU threads exceeds n^2, each thread updates a single matrix entry. Otherwise, each thread processes approximately $n^2/\texttt{thread_count}$ elements.

Coalesced memory access: While exposing parallelism is essential, memory access patterns play a critical role in GPU performance. Efficient execution requires coalesced global memory accesses, where threads within a warp access contiguous memory locations. This allows multiple memory requests to be combined into a single transaction, reducing latency and improving bandwidth utilization.

For the Floyd–Warshall algorithm, coalesced access can be achieved through appropriate thread-to-data mappings. One approach uses a two-level iteration within the kernel: blocks iterate over matrix rows, while threads iterate over columns. The row index is initialized using the block ID and incremented by the grid size, while the column index is initialized using the thread ID and incremented by the block size. This organization ensures that threads within a warp access adjacent memory locations.

An alternative approach uses a single loop in which each thread traverses the matrix using its global thread ID as the starting index and strides by the total number of threads. Although this method also preserves coalesced access, it requires additional modulus and division operations to recover row and column indices, introducing minor overhead. In this work, the former approach is adopted due to its lower overhead and simpler implementation.

B. DPI

This section formulates DPI as a sequence alignment problem, enabling both partial-matching and regular expression (regex)-based DPI as proposed in this work. It then presents the CUDA implementations, including details on memory access design and the use of intrinsic functions.

1) Smith–Waterman Algorithm: The Smith–Waterman algorithm is a dynamic programming method for local sequence alignment. It identifies highly similar subsequences between a query and a target while allowing mismatches, insertions, and deletions. Its local alignment property enables detection of matching regions regardless of their position within the input sequences.

Let the target sequence be $T = (T_1, T_2, \ldots, T_m)$ of length m, and the query sequence be $Q = (Q_1, Q_2, \ldots, Q_n)$ of length n. A scoring matrix $M \in \mathbb{R}^{(n+1)\times(m+1)}$ is defined such that each entry $M_{i,j}$ represents the optimal local alignment score ending at positions Q_i and T_j. Let C, X, I, and D denote the match reward, mismatch penalty, insertion penalty, and deletion penalty, respectively.

The matrix is computed using the recurrence:

$$M_{i,j} = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0, \\ \max \begin{cases} 0, \\ M_{i-1,j-1} + \delta(Q_i, T_j), \\ M_{i-1,j} + D, \\ M_{i,j-1} + I \end{cases} & \text{if } i, j > 0, \end{cases} \tag{2a}$$

where the scoring function δ is defined as:

$$\delta(Q_i, T_j) = \begin{cases} C & \text{if } Q_i = T_j, \\ X & \text{otherwise.} \end{cases} \tag{2b}$$

The first row and column are initialized to zero. The inclusion of zero in the recurrence prevents negative scores from propagating, ensuring that only locally optimal alignments are preserved.

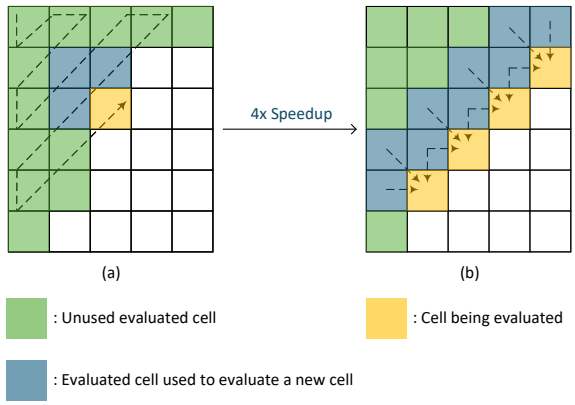

Fig. 1. Wavefront implementation technique for Smith–Waterman algorithm.

After filling the matrix, the maximum entry in M identifies the endpoint of the highest-scoring local alignment. A traceback procedure reconstructs the alignment by following the direction of the maximum contributing term (diagonal, vertical, or horizontal) until a cell with value zero is reached.

2) Extending Smith–Waterman Algorithm for Regex Matching: The Smith–Waterman algorithm does not natively support regex semantics. However, its scoring formulation can be extended to incorporate common regex operators while preserving the dynamic programming structure. This extension enables regex matching while retaining compatibility with hardware accelerators designed for dynamic programming.

A central challenge in this adaptation is maintaining a consistent interpretation of the alignment score. In the standard formulation, the maximum achievable score is bounded by $\min(n, m) \cdot C$. Consequently, a threshold of the form $\alpha \cdot m \cdot C$ corresponds to a fraction α of matching symbols. This interpretation no longer holds when non-literal matches contribute positively to the score.

a) Wildcard Operator $*$: Consider the wildcard operator $*$, representing arbitrary repetition. A naive implementation assigns a match reward to wildcard alignments, i.e.,

$$\delta(Q_i, *) = C.$$

However, this formulation inflates alignment scores because insertions aligned with the wildcard contribute positively. As a result, the alignment score no longer reflects the proportion of literal matches.

To preserve this interpretation, wildcard matches are assigned zero contribution:

$$\delta(Q_i, *) = 0.$$

Under this definition, only literal matches increase the alignment score, while wildcard matches allow flexible alignment without affecting the score.

b) Optional Operator ?: The optional operator ?, representing zero or one occurrence, is incorporated by allowing a single non-penalized mismatch. This is achieved by assigning zero penalty when the target symbol corresponds to ?:

$$\delta(Q_i, T_j) = 0 \quad \text{if } T_j = ?.$$

c) Modified Recurrence: The recurrence relation is modified to incorporate regex-aware transitions:

$$M_{i,j} = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0, \\ \max \begin{cases} 0, \\ M_{i-1,j-1} + \delta(Q_i, T_j), \\ M_{i-1,j} + D, \\ M_{i,j-1} + I \cdot \mathbf{1}_{\{T_j \neq *\}} \end{cases} & \text{if } i, j > 0, \end{cases} \tag{3a}$$

where $\mathbf{1}_{\{\cdot\}}$ is the indicator function. This formulation prevents insertions from contributing to the score when aligned with the wildcard operator.

The scoring function is updated as:

$$\delta(Q_i, T_j) = \begin{cases} 0 & \text{if } T_j \in \{*, ?\}, \\ C & \text{if } Q_i = T_j, \\ X & \text{otherwise.} \end{cases} \tag{3b}$$

d) Generalization to Other Operators: This approach extends to other regex operators by modifying the scoring function so that non-literal matches do not contribute positively to the alignment score. For example, to support the digit character class (i.e., $\backslash\mathtt{d}$ or [0–9]), the scoring function is defined as:

$$\delta(Q_i, T_j) = 0 \quad \text{if } Q_i \in [0\text{-}9] \text{ and } T_j = \backslash\mathtt{d}.$$

Each additional operator introduces conditional logic into the scoring function or recurrence, increasing computational overhead.

3) Problem Formulation: The DPI task is formulated as a set of independent local alignment subproblems, where each subproblem compares a known signature against the packet payload. Let $S = \{s_1, s_2, \ldots, s_n\}$ denote the set of signatures, and let p represent the packet payload. Let $|S| = n$ and define $L(S) = \sum_{k=1}^{n} |s_k|$ as the total length of all signatures.

For each signature $s_k \in S$, a score matrix $m^{(k)} \in \mathbb{N}^{(|p|+1)\times(|s_k|+1)}$ is constructed, where each entry $m_{i,j}^{(k)}$ represents the alignment score between the prefix $p_1 \ldots p_i$ and $s_{k,1} \ldots s_{k,j}$. Let $M = (m^{(1)}, m^{(2)}, \ldots, m^{(n)})$ denote the collection of all such matrices.

The objective is to determine whether there exists a signature $s_k \in S$ and indices $i \in [1, |p|]$, $j \in [1, |s_k|]$ such that:

$$m_{i,j}^{(k)} \geq \alpha \cdot C \cdot |s_k|.$$

In partial matching, $\alpha \in [0, 1]$ represents the fraction of symbols in s_k that must match the payload. In regex matching, α corresponds to the number of literal characters in s_k, excluding contributions from wildcard operators. Consequently, the matching condition becomes:

$$m_{i,j}^{(k)} = \alpha \cdot C.$$

4) CUDA Implementation: The wavefront method depicted in Fig. 1 is a common approach for parallelizing the Smith–Waterman algorithm. This technique utilizes the inherent diagonal dependencies in the scoring matrix, allowing multiple cells to be updated concurrently (up to $min(|T|, |Q|)$ cells). In the context of DPI, signatures are typically short relative to the number of available GPU threads, which leads to poor

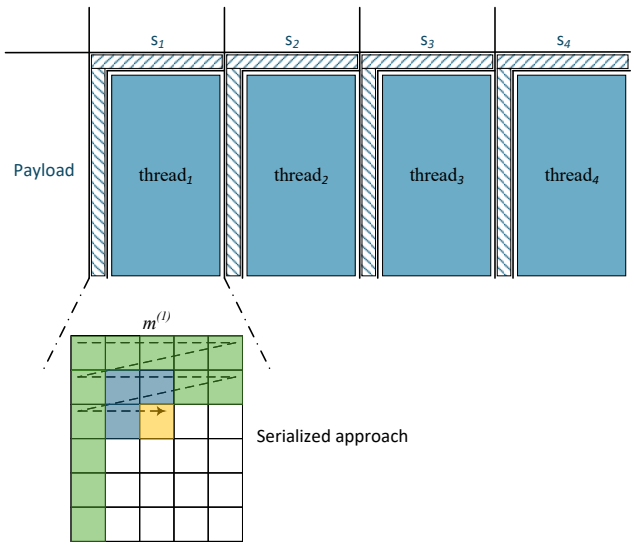

Fig. 2. Signature-level parallelization with one thread per signature.

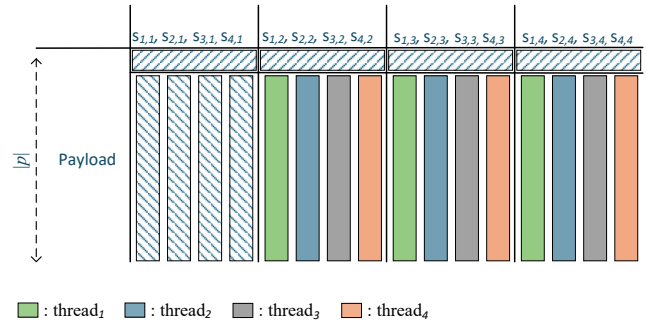

Fig. 3. Memory layout enabling coalesced access across threads.

occupancy if each matrix is processed independently [40]. To better utilize the hardware, the implementation instead parallelizes across signatures: each thread is assigned one signature and computes its full score matrix against the packet payload independently. This design allows many alignment problems to be evaluated concurrently while preserving the exact scoring semantics of the Smith–Waterman recurrence. As shown in Fig. 2, threads and signatures have a one-to-one mapping, yielding a new parallelizing bound equivalent to the hardware thread count. Because the scoring matrix for each signature is computed by a single thread, a serialized computation approach is implemented to reduce the application's complexity.

During execution, each thread computes the matrix associated with its assigned signature and compares the resulting scores against the similarity threshold. When a cell exceeds the threshold, the thread marks the corresponding signature as a potential match. In practice, two reporting strategies are possible. One option is to have each thread write its outcome to a private location and perform a later reduction step. A more lightweight alternative is to use a shared match flag that is updated when any thread detects a hit. Since the goal is to determine whether a candidate signature matches the payload, rather than to count all matches, the second approach is sufficient and reduces synchronization overhead.

Coalesced memory access: While exposing parallelism across threads is necessary to utilize the GPU, overall performance is fundamentally constrained by global memory behavior. In particular, the efficiency of memory transactions depends on whether threads within a warp generate coalesced accesses. A warp achieves full coalescing when its threads access consecutive and properly aligned memory addresses, allowing the hardware to aggregate these requests into a minimal number of memory transactions. This maximizes the bandwidth utilization and minimizes memory latency, which is critical for memory-bound dynamic programming workloads.

To enforce coalesced access, the memory layout is organized as shown in Fig. 3, where $s_{i,j}$ denotes the jth element of the ith signature. Let n be the number of signatures and $L(S)$ the total length of the signature set after padding. The linearized index of each cell $m^x_{i,j}$ is defined as:

$$L(S) \cdot i + n \cdot j + x$$

where x corresponds to the thread identifier within a warp. Under this mapping, consecutive threads access consecutive memory locations along the signature dimension, which is the fastest-varying index.

For a warp consisting of threads $thread_x$ to $thread_{x+31}$, computing $(m^{(k)}_{i,j} \mid k = x, x+1, \ldots, x+31)$ requires three data dependencies for each DP update, which are laid out in contiguous memory regions:

- $L(S)(i-1) + nj + x \;\rightarrow\; L(S)(i-1) + nj + x + 31$,
- $L(S)i + n(j-1) + x \;\rightarrow\; L(S)i + n(j-1) + x + 31$,
- $L(S)(i-1) + n(j-1) + x \;\rightarrow\; L(S)(i-1) + n(j-1) + x + 31$.

These access patterns ensure that all threads in the warp fetch operands from contiguous cache lines, enabling the GPU to service each operand group with a single coalesced transaction. As a result, the three memory reads required per cell update are executed with minimal transaction overhead, significantly improving throughput.

To eliminate irregular access patterns and reduce index computation overhead, all signatures are padded to a uniform length equal to the longest signature. This transforms the memory layout into a dense, regular structure and allows constant-stride access across threads. Consequently, the total length of the signature set becomes $L(S) = n \cdot |s_{\max}|$, where $|s_{\max}|$ is the maximum signature length. This regularization not only simplifies address generation but also guarantees alignment properties required for fully coalesced memory transactions.

Space requirement: The memory organization in Fig. 3 requires $|s_{\max}| \cdot n \cdot |p|$ storage for the full scoring matrices. For example, with a payload size of 1024 bytes ($|p| = 1024$), one million signatures ($n = 10^6$), and a maximum signature length of 32 bytes ($|s_{\max}| = 32$), the total memory requirement reaches approximately 65 GB when using 16-bit cells.

However, full matrix materialization is unnecessary for score computation, as each update depends only on the current row (or column) and its immediate predecessor. Therefore, the implementation maintains only the current and previous

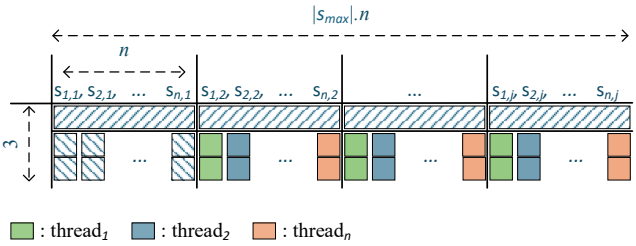

Fig. 4. Sliding-window technique for reducing DP memory usage.

columns, along with an auxiliary initial column to simplify boundary conditions and avoid branching within the kernel. As illustrated in Fig. 4, this sliding-window technique reduces the memory requirement to:

$$3 \cdot |s_{\max}| \cdot n,$$

which lowers the example footprint from 65 GB to 192 MB. This reduction is essential for scaling to large signature sets while preserving high memory throughput and enabling efficient on-device execution.

Specialized DPA functions: The maximum possible alignment score between a signature and a packet payload is bounded by $C \cdot \min(|s_{\max}|, |p|)$, which ensures that 16-bit precision is sufficient to represent all intermediate and final scores without overflow. This enables the use of specialized DPA intrinsic functions to accelerate the updates.

In particular, the $__vimax3_s16x2_relu$ intrinsic is leveraged to fuse multiple operations into a single instruction. This primitive takes three unsigned integer operands a, b, and c, and performs a per-halfword computation of the form $\max(\max(\max(a, b), c), 0)$. As shown in Fig. 5, each 32-bit word encodes two independent 16-bit scores, corresponding to two different signatures. The lower and upper 16-bit lanes are processed concurrently, allowing the DPA to compute two alignment updates in parallel within a single instruction. The resulting 32-bit output preserves this structure, with each halfword containing the updated score for one signature.

Although the use of $__vimax3_s16x2_relu$ requires representing data as 32-bit words, this does not increase the overall memory footprint. Since each word stores two scores, the total number of required cells is effectively reduced by half. Consequently, the number of threads needed to process the workload is also halved, which decreases the number of active

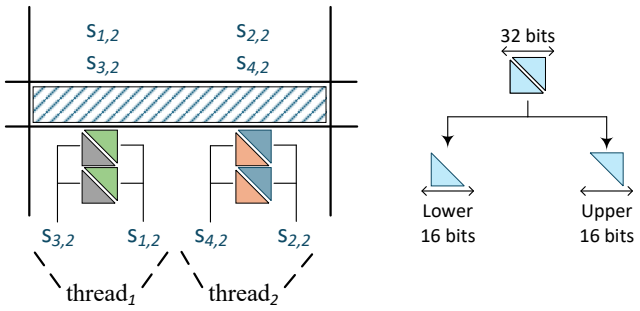

Fig. 5. Halfword DPA operations for parallel score computation.

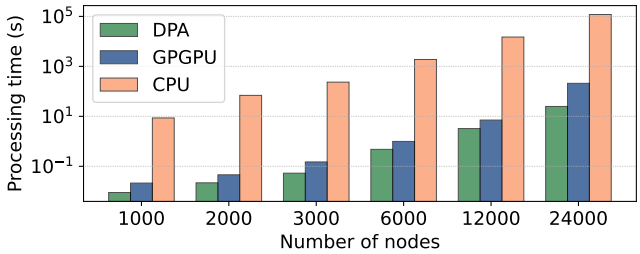

Fig. 6. Processing time comparison for the network resilience application across different topology sizes.

blocks per SM and leads to improved resource utilization and computational efficiency.

VI. Experimental Results

This section evaluates the two applications presented by this paper. For each application, the performance is assessed using the processing time, computational throughput (measured in billion cell updates per second, BCUPS), and energy consumption. Additionally, the impact of coalesced memory access and the influence of different implementation methods are examined. Each application is evaluated on different problem sizes on three hardware targets: NVIDIA H100 (DPA), NVIDIA A100 (GPGPU), and Intel Xeon Silver 4114 (CPU). It is important to note that this CPU is used because the authors have privileged access to it and can accurately measure its energy consumption. Evaluation on a newer-generation CPU showed a consistent 2× performance increase over the CPU used in the evaluation.

A. System One Evaluation

The experimental evaluation of the network resilience application considered topologies with 1K, 2K, 3K, 6K, 12K, and 24K nodes.

1) Processing Time: Fig. 6 presents the processing time required to compute APSP for different network sizes. The results show that the DPA consistently outperforms both the GPGPU and CPU across all configurations. For a topology with 1K nodes, the DPA computes the APSP in 8.9 milliseconds (ms), compared to 334 ms for the GPGPU and 8.6 seconds (s) for the CPU. This corresponds to a speedup of approximately 37× for the DPA over the GPGPU, and 963× for the DPA over the CPU.

The performance advantage of the DPA increases with problem size and reaches its peak at 2K nodes, achieving a 45× speedup over the GPGPU and a 2642× speedup over the CPU. Beyond this point, the speedup relative to the GPGPU gradually declines, reaching 4.6× at 24K nodes. In contrast, the speedup over the CPU decreases to 1941× at 6K nodes and then increases again to 2637× at 24K nodes.

This trend is attributed to increased memory pressure as the problem size grows. The size of the distance matrix increases quadratically with the number of nodes, which results in greater reliance on slower global memory. GPU architectures

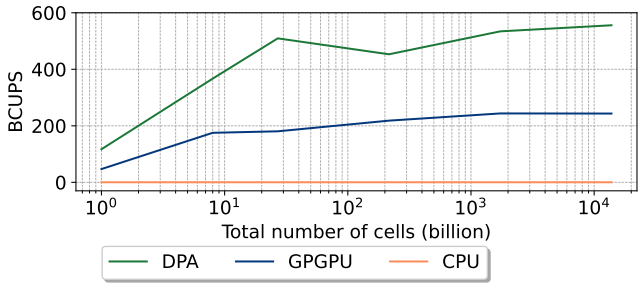

Fig. 7. Computational throughput (BCUPS) achieved for the network resilience application across varying problem sizes.

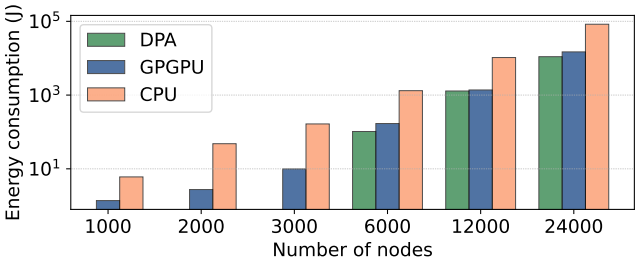

Fig. 8. Energy consumption of the network resilience application across different topology sizes.

mitigate memory latency through massive multithreading, allowing computation to overlap with memory access. This effect benefits the DPA more strongly when compared to the CPU than when compared to the GPGPU. As a result, the relative speedup between the DPA and GPGPU decreases for larger problem sizes, particularly beyond 12K nodes, where memory bandwidth becomes a dominant performance bottleneck.

2) Throughput: Throughput represents the number of dynamic programming cell updates performed per second and is expressed in billions of cell updates per second (BCUPS). It is computed by dividing the problem size, $|V|^3$, by the processing time. This metric provides additional insight into the performance trends observed in the processing time evaluation. The results are shown in Fig. 7.

The first observation is that the throughput of GPU-based platforms, namely the DPA and GPGPU, increases with problem size before reaching a saturation point. The DPA achieves 111.9 BCUPS for topologies with 1K nodes and increases to 305.3 BCUPS for 24K nodes. Similarly, the GPGPU throughput increases from 2.99 BCUPS to 65.99 BCUPS over the same range. In contrast, the CPU maintains a nearly constant throughput of approximately 0.166 BCUPS across all topology sizes. This behavior explains the widening performance gap between the DPA and CPU as the problem size increases.

The second observation is the non-monotonic behavior of the DPA throughput. At 2K nodes, the DPA reaches a peak throughput of 306.5 BCUPS, which decreases to 222.5 BCUPS at 6K nodes. This temporary decline indicates underutilization of computational resources despite the increase in problem size, suggesting that the bottleneck is related to memory access rather than computation. As the problem size continues to grow beyond 6K nodes, the throughput recovers and reaches 306.3 BCUPS at 24K nodes, which is close to the peak value.

This recovery can be attributed to the ability of GPU architectures to hide memory latency through massive multithreading. By launching more virtual threads than the hardware can simultaneously execute, the system ensures that a subset of threads is always performing computations while others wait for data from memory. This overlap between computation and memory access allows the DPA to better utilize its resources at larger problem sizes, resulting in improved throughput.

3) Energy Consumption: This experiment evaluates the energy consumption of each platform using the NVIDIA `NVML` library for the GPUs and `intel-rapl` for the CPU. Energy is computed as the product of average power consumption and execution time. Due to the limited sampling granularity of the measurement tools, energy values corresponding to small problem sizes, where kernel execution times are on the order of tens of milliseconds, are not reliable. As a result, measurements for 1K, 2K, and 3K nodes on the DPA are omitted in the plot shown in Fig. 8.

Despite the superior performance of the DPA in terms of processing time and throughput, it also achieves the lowest energy consumption among all platforms. On average, the DPA consumes 34% less energy than the GPGPU and 9.4× less energy than the CPU. It is important to note that, although the DPA exhibits higher instantaneous power consumption, the reported metric reflects the total energy required to complete the computation, which favors faster execution.

In contrast to the previous experiments, where GPU-based platforms show increasing advantages over the CPU as the problem size grows, the gap in energy consumption between GPU-based implementations and the CPU decreases with larger problem sizes. For example, at 1K nodes, the DPA consumes 12.8× less energy than the CPU, whereas at 24K nodes, this advantage decreases to 7.6×.

To better understand this behavior, the average energy consumed per cell update is analyzed. Since NVML measurements for the DPA are unreliable for some configurations, the GPGPU is used as a proxy due to their similar architectural characteristics. For a topology with 1K nodes, the GPGPU consumes approximately 0.13 nJ per cell update. This value increases to 0.34 nJ and 0.36 nJ for 2K and 3K nodes, respectively, and continues to rise to 0.78 nJ at 6K nodes, 0.79 nJ at 12K nodes, and 1.07 nJ at 24K nodes. In contrast, the CPU maintains a nearly constant energy consumption of approximately 6 nJ per cell update across all problem sizes. This increase in per-cell energy for GPU-based implementations, combined with the stable behavior of the CPU, explains the reduction in the relative energy advantage of the DPA as the problem size increases.

4) Impact of Memory Access Pattern: To evaluate the impact of coalesced memory access on application performance, a non-coalesced version of the implementation is developed, and both processing time and energy consumption are

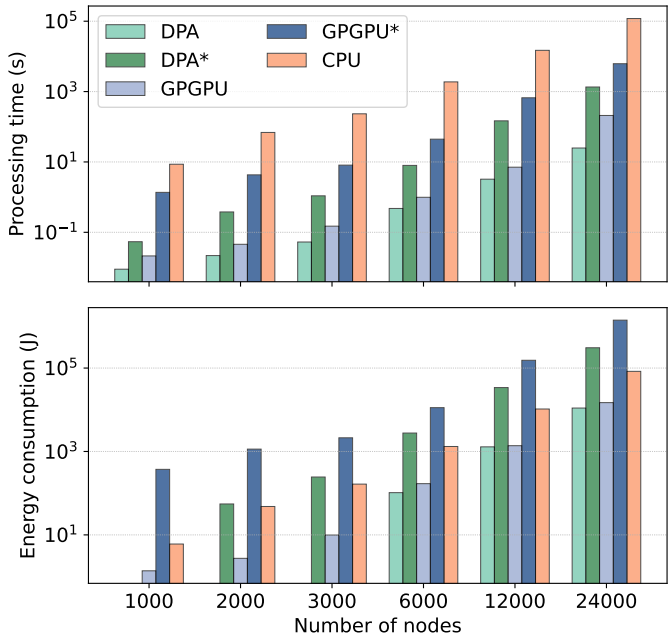

Fig. 9. Impact of coalesced memory access on processing time and energy consumption for GPU-based implementations.

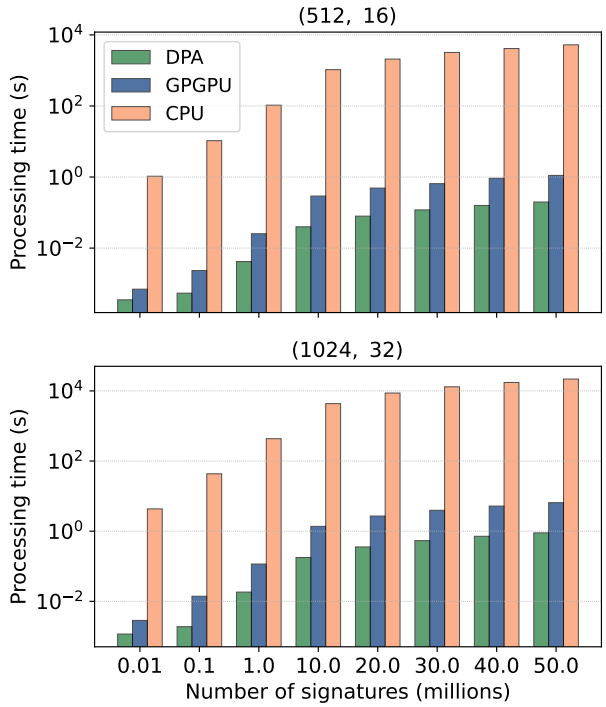

Fig. 10. Processing time of the DPI application for different numbers of signatures and input configurations.

measured. The results are presented alongside the coalesced implementation in Fig. 9, where the symbol $*$ denotes the non-coalesced version.

In terms of processing time, the DPA benefits more from coalesced memory access than the GPGPU. This benefit increases with problem size, as larger workloads require a higher number of memory transactions. For the DPA, coalesced memory access provides a speedup of 6× for 1K nodes, 14.5× for 2K nodes, 9.6× for 3K nodes, 8.2× for 6K nodes, 25.9× for 12K nodes, and 29.8× for 24K nodes. For the GPGPU, the corresponding speedups are 4×, 3.6×, 2.1×, 3.3×, 17.3×, and 29.5×, respectively.

In terms of energy consumption, the GPGPU exhibits a larger relative improvement compared to the DPA. For the DPA, coalesced memory access reduces energy consumption by approximately 26.5× for 6K, 12K, and 24K nodes. For the GPGPU, the reduction is more pronounced, with improvements of 66.4× at 6K nodes, 111.7× at 12K nodes, and 95× at 24K nodes. These results highlight the critical role of memory access patterns in both performance and energy efficiency, particularly for large-scale dynamic programming workloads.

B. System Two Evaluation

The experimental evaluation of the DPI application considers a range of signature set sizes, varying from 10K to 50M. For each configuration, two packet payload sizes (512 and 1024 bytes) and two signature lengths (16 and 32 bytes) are evaluated. In the figures, the notation (x, y) is used to denote these configurations, where x represents the payload size and y represents the signature size.

1) Processing Time: Fig. 10 presents the processing time of the DPI application for varying numbers of signatures. For both configurations, the DPA consistently outperforms the GPGPU and the CPU. Compared to the network resilience application, the DPA exhibits a narrower range of speedup across different problem sizes. In the network resilience application, the speedup over the GPGPU ranges from 4.5× to 45.2×, whereas in the DPI application it ranges from 2× to 7.3×. This difference can be attributed to the nature of the underlying computations. In the network resilience application, the recurrence can be fully expressed using DPA instructions, as it directly maps to the APSP formulation. In contrast, the DPI application requires additional control logic to distribute and manage work across threads, which cannot be entirely captured by DPA primitives. Furthermore, the network resilience application involves multiple kernel launches, while the DPI application is executed within a single kernel, leading to different performance characteristics.

For the $(512, 16)$ configuration, the DPA is the only platform capable of achieving sub-millisecond processing time at moderate scales. It processes 10K and 100K signatures in 350 microseconds (μs) and 549 μs, respectively. This corresponds to speedups of 2× and 4.3× over the GPGPU, and 3012× and 19562× over the CPU. The speedup relative to the GPGPU increases with problem size, reaching a peak of 7.3× at 10M signatures. Beyond this point, the speedup decreases slightly before stabilizing at approximately 5.4× for 30M signatures and above. In contrast, the speedup over the CPU continues to increase with problem size, reaching a steady value of approximately 26498× at 10M signatures.

For the $(1024, 32)$ configuration, similar trends are observed with generally higher speedup values. The speedup over the GPGPU starts at 2.4×, increases to 7.7×, then decreases

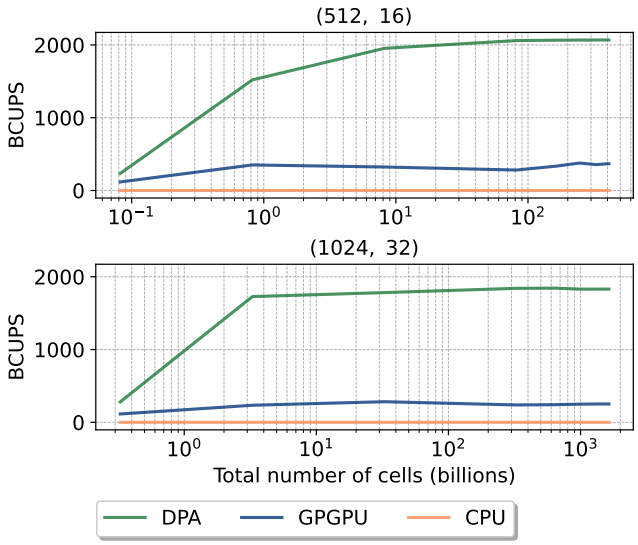

Fig. 11. Computational throughput (BCUPS) achieved for the DPI application across different problem sizes.

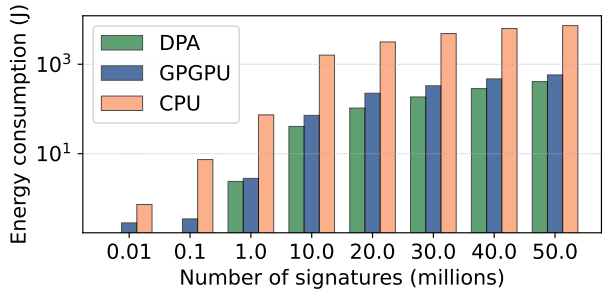

Fig. 12. Energy consumption of the DPI application for varying numbers of signatures.

slightly before stabilizing around 7.4× at 10M signatures. The speedup over the CPU starts at 3700×, increases to 22782×, and then to 23554×, before reaching a steady value of approximately 24337× at 10M signatures. These results confirm that the DPA maintains consistent performance advantages across configurations, while the relative gains depend on the extent to which the workload can be mapped to DPA-supported operations and the volume of memory transactions.

2) Throughput: The throughput results, expressed in BCUPS, are reported in Fig. 11. The top plot corresponds to $(512, 16)$, while the bottom plot corresponds to $(1024, 32)$.

For the $(512, 16)$ configuration, the DPA achieves the highest throughput, exceeding 2000 BCUPS, compared to 380 BCUPS on the GPGPU and 0.077 BCUPS on the CPU. This corresponds to speedups of 5.4× over the GPGPU and 26,701× over the CPU. Both the DPA and GPGPU exhibit increasing throughput as the problem size grows, up to approximately 10M signatures, after which performance plateaus. This behavior indicates saturation of the available computational resources and memory bandwidth.

For the $(1024, 32)$ configuration, the same performance ordering is observed, with lower absolute throughput due to the larger scoring matrices. The DPA reaches a peak throughput of 1830 BCUPS, while the GPGPU achieves 280 BCUPS and the CPU remains around 0.075 BCUPS. These results correspond to speedups of 6.5× over the GPGPU and 24,400× over the CPU. Similar to the smaller configuration, throughput stabilizes as the problem size increases, but at a lower number of signatures. In this configuration, throughput plateaus around 100K signatures, indicating that the available hardware resources are fully utilized at this scale.

Compared to the network resilience application, the DPI application achieves significantly higher throughput for several reasons. First, the network resilience implementation requires frequent interaction between the host and the device, as computing APSP involves $|V|$ kernel launches. In contrast, the DPI application can be executed with a single kernel launch, provided that the scoring matrices fit in GPU memory, reducing host-device communication overhead.

Second, the DPI application exposes substantially higher parallelism. The total number of cell updates is $|s_{\max}| \cdot n \cdot m$, of which $|s_{\max}| \cdot n$ cells can be computed independently at each step. In contrast, the network resilience application allows only $|V|^2$ independent computations out of the $|V|^3$ total updates, limiting parallel efficiency. Finally, the DPI formulation enables the use of halfword DPA intrinsics, effectively doubling computational density and reducing memory traffic, as each matrix element encodes the scores of two signatures.

3) Energy Consumption: In this experiment, the two configurations show similar energy consumption patterns. For this reason, only the $(512, 16)$ configuration is reported. For signature counts of $10K$, $100K$, and $1M$, the `NVML` library failed to capture the energy consumption on the DPA. The results of this experiment are depicted in Fig. 12.

Similar to the network resilience application, the DPA achieves the lowest processing time and the highest throughput while consuming lower energy than the GPGPU and the CPU. For the $(512, 16)$ configuration, both the DPA and GPGPU exhibit significantly lower energy consumption than the CPU. At 10M signatures, the DPA consumes 4.94 J, compared to 9.73 J for the GPGPU and 1597.8 J for the CPU. This corresponds to energy reductions of 2× and 323× relative to the GPGPU and CPU, respectively. As the number of signatures increases, energy consumption grows proportionally across all platforms; however, the DPA consistently maintains the lowest energy profile, achieving approximately 3× and 273× lower energy consumption than the GPGPU and CPU, respectively, at 50M signatures.

4) Impact of DPA Halfword Intrinsic Functions: This experiment evaluates the performance gain achieved by the DPA by comparing the processing time of the H100 GPU with and without the use of DPA halfword intrinsic functions. The results are shown in Fig. 13. As both configurations exhibit similar trends, only the $(512, 16)$ case is presented.

The speedup is lowest at 10K signatures, with a value of 14%, as the problem size is too small to fully utilize the DPA capabilities. As the workload increases to 100K signatures, the speedup rises to 71%, reflecting improved hardware utilization. The maximum speedup is achieved at 10M signatures, reach-

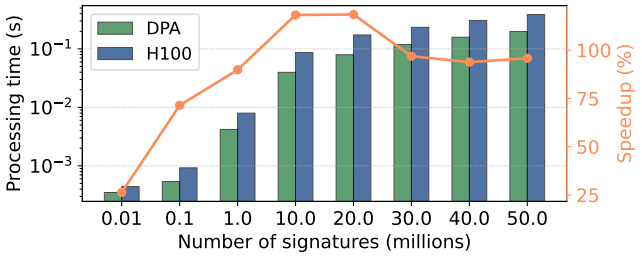

Fig. 13. Impact of DPA intrinsic functions on processing time with respect to raw H100 performance.

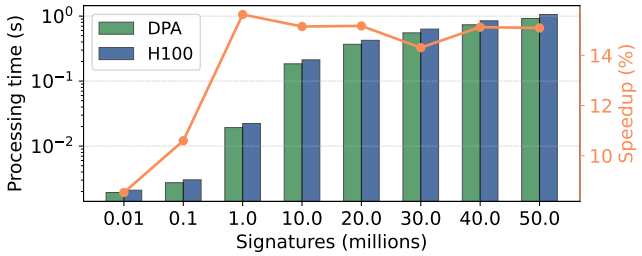

Fig. 14. Impact of enabling regex-based matching on processing time.

ing 118%, demonstrating the effectiveness of DPA intrinsics in accelerating the baseline H100 execution.

Beyond this point, the speedup decreases slightly and stabilizes around 94%. This behavior indicates that, at large problem sizes, memory transactions increasingly dominate execution time, limiting the benefits of additional computational acceleration. Consequently, the massive parallelism of the GPU is insufficient to fully hide memory access latency, reducing the relative advantage of DPA intrinsics.

5) Impact of Enabling Regex Matching: Fig. 14 evaluates the impact of incorporating regex-based matching into the Smith–Waterman algorithm. Although enabling regex introduces additional computational overhead due to the extended scoring logic, the DPA maintains a clear performance advantage over the H100 baseline. The observed speedup remains significant across all problem sizes, indicating that the proposed regex extension can be supported efficiently without compromising scalability. These results demonstrate the practicality of extending dynamic programming-based DPI with more expressive matching capabilities.

A comparison of the speedup achieved by the DPA over the H100 baseline, with and without regex support, further highlights the overhead associated with incorporating regex semantics. However, the primary objective of this work is to demonstrate the feasibility of leveraging domain-specific accelerators to enhance network management operations, including failure recovery and traffic analysis. Future work will focus on optimizing the integration of regex semantics into the Smith–Waterman algorithm.

VII. Conclusion

This paper presents a unified framework for accelerating network management applications, specifically network resilience and deep packet inspection (DPI), using Dynamic Programming Accelerators (DPAs). By leveraging domain-specific hardware support and massive parallelism, the proposed approach improves the efficiency of dynamic programming workloads that underpin network management operations. Two representative applications, network resilience and DPI, are considered to demonstrate how DPAs can enhance both network reliability and traffic visibility in modern networks. The experimental evaluation, conducted across DPA, general-purpose graphics processing units (GPGPUs), and CPUs, considers key performance metrics including processing time, computational throughput, and energy consumption. The results indicate that DPAs can reduce computation time, increase throughput, and improve energy efficiency, thereby enabling faster failure recovery and more efficient traffic analysis.

References

[1] X. Wang, Q. Zhang, J. Ren, S. Xu, S. Wang, and S. Yu, "Toward efficient parallel routing optimization for large-scale SDN networks using GPGPU," *Journal of Network and Computer Applications*, vol. 113, pp. 1–13, 2018.

[2] A. Shanbhag, S. Madden, and X. Yu, "A study of the fundamental performance characteristics of GPUs and CPUs for database analytics," in *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, 2020, pp. 1617–1632.

[3] K.-E. Berger and F. Galea, "An efficient parallelization strategy for dynamic programming on GPU," in *2013 IEEE International Symposium on Parallel & Distributed Processing, Workshops and Phd Forum*. IEEE, 2013, pp. 1797–1806.

[4] Z. Zhu, S. Gupta, N. Pivaro, S. R. Deshpande, and M. Canova, "A GPU implementation of a look-ahead optimal controller for eco-driving based on dynamic programming," in *2021 European Control Conference (ECC)*. IEEE, 2021, pp. 899–904.

[5] W. Kool, H. van Hoof, J. Gromicho, and M. Welling, "Deep policy dynamic programming for vehicle routing problems," in *International conference on Integration of Constraint Programming, Artificial Intelligence, and Operations Research*. Springer, 2022, pp. 190–213.

[6] NVIDIA, "Nvidia hopper architecture," 2025, [Online]. Available: https://tinyurl.com/4da6njpe.

[7] W. Luo, R. Fan, Z. Li, D. Du, Q. Wang, and X. Chu, "Benchmarking and dissecting the nvidia hopper GPU architecture," in *2024 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 2024, pp. 656–667.

[8] A. Mazloum, E. Kfoury, J. Gomez, and J. Crichigno, "A survey on rerouting techniques with P4 programmable data plane switches," *Computer Networks*, vol. 230, p. 109795, 2023.

[9] T. Holterbach, S. Vissicchio, A. Dainotti, and L. Vanbever, "Swift: Predictive fast reroute," in *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*, 2017, pp. 460–473.

[10] A. Mazloum, S. Elizalde, S. Choueiri, E. Kfoury, J. Gomez, A. AlSabeh, and J. Crichigno, "Optimizing network resilience using domain-specific hardware accelerator for dynamic programming," in *Proceedings of the SC'25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2025, pp. 860–865.

[11] J. Drew, T. Moore, and M. Hahsler, "Polymorphic malware detection using sequence classification methods," in *2016 IEEE Security and Privacy Workshops (SPW)*. IEEE, May 2016, pp. 81–87.

[12] V. Naidu and A. Narayanan, "Needleman-wunsch and smith-waterman algorithms for identifying viral polymorphic malware variants," in *2016 IEEE 14th Intl Conf on Dependable*. IEEE, 2016, pp. 326–333.

[13] A. Dinh, D. Brill, Y. Li, and W. He, "Malware sequence alignment," in *2016 IEEE International Conferences on Big Data and Cloud Computing (BDCloud)*. IEEE, 2016, pp. 613–617.

[14] A. Mazloum, "Source codes," https://github.com/AliDMazloum/Toward_Optimizing_Networking_and_Cybersecurity_Applications.

[15] G. E. Moore *et al.*, "Cramming more components onto integrated circuits," 1965.

[16] ——, "Progress in digital integrated electronics," in *Electron devices meeting*, vol. 21. Washington, DC, 1975, pp. 11–13.

[17] E. F. Kfoury, S. Choueiri, A. Mazloum, A. AlSabeh, J. Gomez, and J. Crichigno, "A comprehensive survey on smartnics: Architectures, development models, applications, and research directions," *IEEE Access*, vol. 12, pp. 107 297–107 336, 2024.

[18] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *Proceedings of the April 18-20, 1967, spring joint computer conference*, 1967, pp. 483–485.

[19] R. Kawashima, "Software physical/virtual rx queue mapping toward high-performance containerized networking," *IEEE Transactions on Network and Service Management*, vol. 18, no. 1, pp. 687–700, 2021.

[20] E. Kfoury, J. Crichigno, and E. Bou-Harb, "P4bs: Leveraging passive measurements from p4 switches to dynamically modify a router's buffer size," *IEEE Transactions on Network and Service Management*, vol. 21, no. 1, pp. 1082–1099, 2024.

[21] F. Ihle, S. Lindner, and M. Menth, "P4-psfp: P4-based per-stream filtering and policing for time-sensitive networking," *IEEE Transactions on Network and Service Management*, vol. 21, no. 5, pp. 5273–5290, 2024.

[22] E. F. Kfoury, J. Crichigno, and E. Bou-Harb, "An exhaustive survey on P4 programmable data plane switches: Taxonomy, applications, challenges, and future trends," *IEEE access*, vol. 9, pp. 87 094–87 155, 2021.

[23] N. Kröger, H. Harkous, F. Mehmeti, and W. Kellerer, "Performance modeling and analysis of p4 programmable devices with general service times," *IEEE Transactions on Network and Service Management*, vol. 21, no. 4, pp. 4543–4562, 2024.

[24] M. Je, "[CUDA] Warps and SIMT," 2024, [Online]. Available: https://tinyurl.com/3jsx9aey.

[25] NVIDIA, "Tuning cuda applications for Hopper GPU architecture," 2025, [Online]. Available: https://tinyurl.com/msdfc2ps.

[26] ——, "CUDA C++ programming guide," *NVIDIA, July*, 2020.

[27] F. He and E. Oki, "Main and secondary controller assignment with optimal priority policy against multiple failures," *IEEE Transactions on Network and Service Management*, vol. 18, no. 4, pp. 4391–4405, 2021.

[28] Y. Hirano, F. He, T. Sato, and E. Oki, "Preventive start-time optimization to determine link weights against probabilistic link failures," *IEEE Transactions on Network and Service Management*, vol. 18, no. 2, pp. 2275–2293, 2020.

[29] J. Cheng, D. T. Nguyen, and V. K. Bhargava, "Resilient edge service placement under demand and node failure uncertainties," *IEEE Transactions on Network and Service Management*, vol. 21, no. 1, pp. 558–573, 2024.

[30] M. Sasabe and T. Hara, "Capacitated shortest path tour problem-based integer linear programming for service chaining and function placement in nfv networks," *IEEE Transactions on Network and Service Management*, vol. 18, no. 1, pp. 104–117, 2021.

[31] E. Teskeredžić, K. Karahodžić, and N. Nosović, "Comparison of the non-blocked and blocked floyd-warshall algorithm with regard to speedup and energy saving on an embedded GPU," in *2020 19th International Symposium INFOTEH-JAHORINA (INFOTEH)*, 2020, pp. 1–5.

[32] G. J. Katz and J. T. Kider, "All-pairs shortest-paths for large graphs on the GPU," in *Proceedings of the 23rd ACM SIGGRAPH/EUROGRAPHICS symposium on Graphics hardware (GH'08)*, no. 180, 2008, pp. 47–55.

[33] P. Harish and P. J. Narayanan, "Accelerating large graph algorithms on the GPU using cuda," in *International conference on high-performance computing*. Springer, 2007, pp. 197–208.

[34] A. V. Aho and M. J. Corasick, "Efficient string matching: an aid to bibliographic search," *Communications of the ACM*, 1975.

[35] Giorgos Vasiliadis, et al., "Gnort: High performance network intrusion detection using graphics processors," in *International workshop on recent advances in intrusion detection*. Springer, 2008, pp. 116–134.

[36] J. Peng, H. Chen, and S. Shi, "The GPU-based string matching system in advanced AC algorithm," in *2010 10th IEEE International Conference on Computer and Information Technology*. IEEE, 2010, pp. 1158–1163.

[37] N.-P. Tran, M. Lee, S. Hong, and M. Shin, "Memory efficient parallelization for aho-corasick algorithm on a GPU," in *2012 IEEE 14th International Conference on High Performance Computing and Communication*. IEEE, 2012, pp. 432–438.

[38] C.-H. Lin, C.-H. Liu, L.-S. Chien, and S.-C. Chang, "Accelerating pattern matching using a novel parallel algorithm on GPUs," *IEEE Transactions on Computers*, vol. 62, no. 10, pp. 1906–1916, 2012.

[39] S. Soroushnia, M. Daneshtalab, J. Plosila, and P. Liljeberg, "Heterogeneous parallelization of aho-corasick algorithm," in *8th International Conference on Practical Applications of Computational Biology & Bioinformatics (PACBB 2014)*. Springer, 2014, pp. 153–160.

[40] C.-L. Hung, C.-Y. Lin, and P.-C. Wu, "An efficient GPU-based multiple pattern matching algorithm for packet filtering," *Journal of Signal Processing Systems*, vol. 86, no. 2, pp. 347–358, 2017.